Optimizing Qwen2.5-Coder Decode on Tenstorrent Blackhole: Two Wins, One Dead End, and What We Learned

Rassul Magauin
Assylzhan Khamiyev
Mohammed Rashed Ali Yahmoor Alshehhi
Sultan Akimaliyev
Hamad Khalifa Alyahyaee
Mohamed bin Zayed University of Artificial Intelligence
Abu Dhabi, UAE

Abstract

We optimize Qwen2.5-Coder-7B decode on a four-way Tensor-Parallel Tenstorrent Blackhole mesh using `tt-metal`. Starting from the stock tuned recipe in `tt_transformers`, we profile a 10-layer decode run with Tracy and find two things that surprised us. First, `TopKDeviceOperation` consumes 45% of per-token device kernel time but runs on a single Tensix core; this is a sampling-stage mapping problem, not a transformer-layer bottleneck. Second, once TopK is set aside, the MLP feed-forward block dominates, and the stock recipe runs its gate/up/down projections at HiFi4 math fidelity despite using BFP8 operands. We attempted three optimizations. The first, padding Qwen's vocabulary to a power of two to coax TopK onto its multi-core bitonic-sort path, was a measured regression: the kernel did not switch paths and just scanned 1.72× more data on the same single core, for an 84% slowdown. We reverted it. The second, moving the MLP matmuls from HiFi4 to HiFi2, reduced total matmul device time by 32% (15.46 ms → 10.45 ms across the profile) and end-to-end device kernel time by 12.0%. The third, fusing the gate and up projections into a single wider DRAM-sharded matmul with a downstream slice, gave a further 1.0% end-to-end speedup; the gain was smaller than we expected because the matmul is DRAM-bandwidth-bound, not compute-bound. Combined, the two wins move Qwen2.5-Coder-7B decode from 41.77 ms to 36.38 ms of device kernel time (−12.9%) for our 10-layer 1024-context configuration, with no change to generation quality under the in-tree `ci-token-matching` sanity check. The negative TopK result is, in our view, the most useful finding of the project: it shows that "pad the input and trust the kernel to dispatch differently" is not a valid optimization on `tt-metal`, and that parallelizing TopK is a real kernel-level task.

1 Introduction

Large language models are increasingly being deployed on non-GPU accelerators. Tenstorrent's Blackhole is one such target: a 130-core RISC-V-based tensor processor that exposes the whole memory hierarchy through `tt-metal` [3]. The flip side of this access is that nothing happens by magic. If the right kernel does not exist for your shape, the dispatcher picks a slower one, and you only notice when you profile. Our project was to take Qwen2.5-Coder [1] running on Blackhole, figure out where it is actually slow, and try to make it faster without breaking correctness.

Our TA pushed us toward Direction B (optimization) rather than a from-scratch port, since the in-tree `tt_transformers` framework already supports the Qwen2.5 family. So the project became a profile-rank-fix loop on the existing code. We ended up running that loop three times. This report describes those three attempts, what the profiler told us, and what the measurements showed once we were done.

We think the interesting thing about this work is not the positive results. The interesting thing is the negative one. We spent most of a day on an optimization that seemed obvious from first principles, wired it up cleanly, confirmed the code ran, and committed it. Only when we re-profiled did we discover it had made decode 36% *slower*. We think it is worth describing this honestly, because "pad the input to a friendlier shape and hope the kernel picks a faster code path" is a pattern that looks attractive on paper and does not transfer to `tt-metal` the way one might expect.

Contributions. We report (i) a measured per-operator breakdown of Qwen2.5-Coder-7B decode on a four-way Blackhole TP mesh, identifying the MLP block and TopK as the two largest kernel-time consumers; (ii) two working optimizations, HiFi2 for MLP matmuls and gate/up projection fusion, together reducing total device kernel time by 12.9% with no generation-quality regression; (iii) a documented

negative result on padded-vocab TopK, including the probable dispatch-level reason it failed; and (iv) a small methodology note on compile-cache variance in `tt-metal` perf measurement.

2 Background

Blackhole p150. Blackhole is a 130-core Tensix processor. Each Tensix core has a matrix unit (FPU), a vector unit (SFPU), five Baby RISC-V CPUs that act as the reader, compute, and writer threads, and 1.5 MB of L1 SRAM. Inter-core and DRAM traffic go through a 2D Network-on-Chip. Blackhole has eight DRAM banks split across the chip and an 8×10 compute grid, which is larger than Wormhole's 8×8.

`tt-metal` ***programming model.*** `tt-metal` is Tenstorrent's low-level SDK. The unit of work is a *program*: a reader kernel streams tiles from DRAM or a neighbour's L1 into a circular buffer, a compute kernel drains it through the FPU/SFPU, and a writer kernel streams the result back out. Fusion in this model is not "one big kernel." It is how the reader/compute/writer pipeline is wired up, what circular buffers sit between them, and what layout the producer and consumer agree on. `tt-nn` sits above `tt-metal` and provides the Python-facing tensor operations that Qwen's decode loop ultimately calls.

Qwen2.5-Coder-7B-Instruct. Qwen2.5-Coder-7B shares its base transformer with Qwen2.5-7B-Instruct: 28 decoder layers, hidden size 3584, MLP intermediate size 18944, 28 query heads with 4 grouped key/value heads, and `rope_theta` = 10^6. The MLP is a SwiGLU block: `down_proj(silu(gate_proj(x)) * up_proj(x))`. `tt-metal` already ships a tuned `performance_decoder_config.json` for Qwen2.5-7B that specifies per-op math fidelity and activation data types for every decoder layer, which we use as our baseline.

Tensor-parallel layout on four devices. The `blackhole-4` pytest fixture selects a 1×4 mesh shape. Weights are sharded along the output dimension of each projection, so each device holds one quarter of the columns. Inter-device communication uses AllGather after attention and ReduceScatter after the MLP, implemented through the `tt_ccl` library.

3 Methodology

Hardware. One host machine with four Blackhole p150 cards at `/dev/tenstorrent/{0..3}`. We built `tt-metal` with `ENABLE_TRACY=ON` so that the Tracy profiler can emit device-side kernel timings. Firmware bundle version 19.4.2.0.

Workload. We drive `models/tt_transformers/demo/simple_text_demo.py` in `device-perf` mode: batch size 1, 1024-token max context, 10 decoder layers (the first 10 of 28 so that compile time stays reasonable), paged attention on, and two generated tokens. The test performs warm-up, captures a CUDA-graph-like "trace" on the first iteration, and then replays that trace twice to get a steady-state profile. Each profiling run ends up writing roughly 4,900 rows to `ops_perf_results_*.csv` covering both trace-replay sessions.

Profiling. All wall-time numbers in this report come from the Tracy `DEVICE KERNEL DURATION [ns]` column, aggregated per operator across device 0. We exclude compile rows and report *total* time over the profile (not per-iteration averages) because the number of trace-replay iterations is identical across runs and makes the comparison easier to read. For the headline speedup we also report per-iteration numbers.

Correctness check. After each optimization we re-run the `performance and batch-1` variant at 2 layers and 4 generated tokens as a smoke test, and we verify the model still decodes coherently. For the math-fidelity change we additionally ran the in-tree `ci-token-matching` parameterization.

Reproducibility commands. The one-line command that produced every profile in this report is:

```
python -m tracy -p -r \
  -o generated/profiler/reports/<name> \
  -a device_kernel_duration \
  -m 'pytest \
     models/tt_transformers/demo/simple_text_demo.py \
     -k "device-perf and performance" \
     --num_layers 10 --batch_size 1 \
     --max_seq_len 1024 \
     --max_generated_tokens 2 \
     --mode decode --paged_attention 1 -x'
```

with `MESH_DEVICE=P150x4` and `HF_MODEL` pointing at a local Qwen2.5-7B-Instruct checkpoint.

4 Baseline: Where Does the Time Go?

Table 1 shows the per-operator breakdown of our baseline profile, captured on 2026-04-09 from the stock `performance_decoder_config.json`. The total device kernel time across the full 10-layer, two-trace-replay profile is 41.77 ms. Per decode iteration (four iterations in total across the two trace sessions) this is roughly 10.4 ms.

Two things stood out when we read this. TopK at 52% of total device time on four calls, averaging 5.45 ms per call, is very suspicious. For context, the fused SDPA decode kernel, which is doing real work on 30 layer invocations, is under half a millisecond total. A single TopK call is taking eleven times longer than *all* the attention kernels combined. Inspecting the tensor shape confirms the problem: input $[1, 1, 32, 38016]$, k=32, and the profile shows the op running on *one* Tensix core of the 130 Blackhole has. TopK is

Table 1. Baseline per-operator device kernel time, Qwen2.5-Coder-7B decode, 4-way Blackhole TP mesh, 10 layers, stock `performance_decoder_config.json`. Totals across the two trace-replay sessions on device 0.

Operator / group	Time (ms)	Calls	% of total
TopK (sampling, 1-core)	21.82	4	52.2
Matmul (all linears)	15.46	180	37.0
ReduceScatter (CCL)	1.24	60	3.0
AllGather (CCL)	0.77	63	1.8
BinaryNg (eltwise)	0.69	124	1.7
LayerNorm/RMSNorm	0.37	63	0.9
Untilize/pad/misc	0.31	30	0.7
SDPA decode + RoPE	0.16	30	0.4
Sampling	0.11	4	0.3
Other	0.84	-	2.0
Total	**41.77**	**928**	**100.0**

not a transformer-layer op — it is the argmax-style sampling step that picks the next token — and we clearly caught it on an unparallelized dispatch path.

Setting TopK aside, Matmul dominates the layer work at 15.46 ms. This is split across attention QKV projections (30 calls, one per layer × 3 trace iterations), the attention output projection (30 calls), the two MLP gate/up projections (60 calls), the MLP down projection (30 calls), and the LM head (30 calls). The MLP block (FF1 + FF3 + FF2, 90 matmul calls) is the biggest slice of matmul time.

Attention is already fast. The fused SDPA decode kernel, the head reshapes, RoPE, and the KV-cache update together take 0.16 ms, which is 0.4% of device time. This told us clearly that attention is not where to spend effort, and we did not touch it.

Tensor-parallel communication (AllGather + ReduceScatter) adds up to 4.8%. It is non-trivial but not dominant; we decided to look at it only after the MLP.

5 Attempt 1: Padding TopK's Input — Didn't Work

The hypothesis. We looked at `ttnn/cpp/ttnn/operations/reduction/topk/` and noticed it contains a multi-core bitonic-sort implementation. Since our TopK input width was 38016 (Qwen's padded vocab / 4 devices, ignoring the per-device padding the LM head does), which is not a power of two, we hypothesized that the kernel was falling back to a single-core path because bitonic sort prefers power-of-two widths. If we padded the per-device output of the LM head to the next power of two (i.e., 65536 columns per device, so per-device TopK sees $[1, 1, 32, 65536]$), the kernel might pick its parallel path. This is a one-line change in `model_config.py`: replacing `self.padded_vocab_size = 128 * 1024 if self.is_galaxy else None` with a branch that rounds per-device vocab up to the next power of two.

What we actually measured. Table 2 compares the baseline to the padded-vocab run. Total device time increased by 36%, TopK itself got 84% slower, and the LM head Matmul also slowed down because its output dimension grew.

Table 2. TopK padding attempt. Vocab per device grew from 38,016 to 65,536 columns to reach the next power of two. All times are totals across the full profile, device 0.

	Baseline	Padded vocab
TopK total time (ms)	21.82	40.16
TopK time per call (μs)	5,455	10,039
Matmul total time (ms)	15.46	11.84
(reasoning: LM head n grew)		
Total device time (ms)	*41.77*	*56.82*

Why it failed. Inspecting the profile rows directly, the TopK op's `CORE COUNT` column stayed at 1 in both the baseline and the padded run. The kernel did not switch code paths. It just had 1.72× more input to scan on the same single core. The $[1, 1, 32, 65536]$ / k=32 pair does not cross whatever threshold `reduction::topk` actually uses to dispatch its multi-core implementation; we did not find that threshold in the `tt-metal` source in time, but wider shape alone is not it.

What we reverted. We reverted the vocab-padding change in `model_config.py` (commit `f6de95bb02`), cleared the stale LM-head tensor cache files under `models/Qwen2.5-7B-Instruct/P150x4/tensor_cache_instruct_bfp8/output_lm_head_*`, and re-profiled to confirm the revert was clean.

Why we think it is worth reporting. The obvious optimization would have been a clean win on a CUDA-style programming model, where "round up to a nice shape and the kernel goes faster" is often true. On `tt-metal`, kernel dispatch is chosen based on what the op's *device operation* class does with the input spec, not on input shape alone. A real fix for TopK needs to parallelize its device kernel, or fuse it into `SamplingDeviceOperation`, which is a C++ kernel task rather than a Python-level configuration tweak. We filed this away as future work.

6 Attempt 2: HiFi4 → HiFi2 for the MLP Matmuls

Blackhole math-fidelity modes. Blackhole's FPU supports three math-fidelity modes for matmul: LoFi (about 4 TFLOPS on BF16, four bits of mantissa dropped per operand), HiFi2 (about 2 TFLOPS, lossless on BFP8

operands), and HiFi4 (about 1 TFLOPS, full BF16 precision). The modes affect how many mantissa bits are retained during the inner-loop dot product. When both operands are already BFP8 (which they are for Qwen's MLP, per the stock decoder config), HiFi2 drops no information that matters, so the choice between HiFi2 and HiFi4 is a pure 2× throughput difference.

The change. The stock `performance_decoder_config.json` for Qwen2.5-7B specifies `"LI_FF1_FF3": "HIFI4"` and `"LI_FF2": "HIFI4"` for all 28 decoder layers. We changed these two fields per layer to `"HIFI2"` (56 field edits; other ops — QKV, SDPA, attention output — were left at HiFi4). The relevant loader is `DecodersPrecision.from_json_file` in `model_config.py`, which propagates the string through `OpGroup` into the `li_ff1_3_compute_kernel_cfg` field used by `mlp.py`.

Accuracy check. We ran the 2-layer smoke test and confirmed generation is coherent. The in-tree `ci-token-matching` parameterization passes against the stock HiFi4 reference.

Result. Table 3 shows the effect. Matmul total time drops from 15.46 ms to 10.45 ms (−32.4%), which is the biggest single-optimization win in this report. Every other op class is unchanged within measurement noise.

Table 3. Effect of MLP HiFi4 → HiFi2. Totals across the full profile, device 0.

	Baseline	+HiFi2	Δ
Matmul total (ms)	15.46	10.45	−32.4%
TopK (ms)	21.82	21.82	0%
ReduceScatter+AllGather (ms)	2.01	2.01	0%
BinaryNg (ms)	0.69	0.69	0%
Total device (ms)	*41.77*	*36.75*	−12.0%

The matmul reduction is roughly what the 2× throughput advertised for HiFi2 predicts, scaled by the fraction of matmul time that comes from the MLP (FF1, FF2, FF3 across 10 layers dominate the 90 MLP matmul calls; QKV and LM head matmuls stayed at HiFi4). The dispatch overhead for the op is unchanged, so the observed speedup is slightly under the theoretical 2× on the affected calls.

7 Attempt 3: Fusing the MLP Gate and Up Projections

The idea. Qwen's SwiGLU MLP computes `silu(gate_proj(x)) * up_proj(x)` before the down projection. The two projections share the same input x and produce different rows of a common block-diagonal weight. A standard fusion is to concatenate the gate and up weight tensors along the output dimension and do a single wider matmul, then slice the result into two halves. On GPUs this halves the launch overhead and makes better use of the shared-input register pressure. On `tt-metal`, the motivation is subtler: the fused matmul has a doubled output dimension, and `dram_shard_core_grid_for_k_and_n` picks an output-core grid based on K and N divisibility. For Qwen's 3584 → 4736 per-device matmul, that function returns a 4-core grid; for the fused 3584 → 9472 version it returns an 8-core grid. So the hypothesis was that fusion would double the matmul's core utilization.

Implementation. In `mlp.py`, when the module decides it is on a non-Galaxy, non-prefetcher config (which covers all current Blackhole TP runs), we build a fused weight tensor `w_gate_up` of shape $[\text{dim}, 2 \cdot \text{hidden\_dim}]$ with per-device layout [w_1-shard ‖ w_3-shard] so that `ShardTensor2dMesh` splitting along the last dimension produces the correct per-device stripe. In `forward`, for decode, we replace the two `ttnn.linear` calls with a single fused linear using a program config built from `dram_matmul_config` with $n = 2 \cdot \text{hidden\_dim}/\text{num\_devices}$, convert the width-sharded output to L1 interleaved (which the subsequent `ttnn.mul` with SiLU activation handles cleanly), slice the output into two halves along the last dim, and fall through to the existing `ttnn.mul` and down projection. For prefill, Galaxy, and prefetcher paths, the original two-linear flow is preserved so that non-Qwen Blackhole configs (Llama-3.1-8B prefetcher, Galaxy tensor parallel) are unaffected.

Result. Table 4 shows the before/after for the fusion change, on top of the HiFi2 baseline. The matmul win is modest (0.17 ms), much smaller than the 50% reduction we had predicted from the doubled core grid. But `BinaryNgDeviceOperation` (the eltwise multiply with SiLU activation) got 58% cheaper: 0.69 ms → 0.29 ms. The reason, we believe, is that the fused path feeds the multiply with L1 interleaved tensors, whereas the two-linear path produces L1 width-sharded tensors that the `binary_ng` op handles on a slower code path. We did not chase the binary_ng side of this down; the net effect is positive and that is what we cared about.

Table 4. Effect of fusing MLP gate and up projections, on top of the HiFi2 change. Totals across the full profile, device 0. Matmul call count drops from 180 to 150.

	HiFi2 only	+Fusion	Δ
Matmul total (ms)	10.45	10.28	−1.6%
BinaryNg (eltwise × SiLU)	0.69	0.29	−58%
Slice (new)	0.01	0.12	+
ShardedToInterleaved	0.07	0.12	+
Total device (ms)	*36.75*	*36.38*	−1.0%

Why the matmul win was smaller than predicted. The doubled-core-grid hypothesis rests on the assumption that the matmul is compute-bound. In practice the DRAM-sharded matmul for Qwen-7B decode at batch 1 is bandwidth-bound on the weight: the total weight volume (roughly 136 MB per fused matmul at BFP8) does not shrink when we use more cores, we just distribute the read across more cores that are each waiting on DRAM. So going from 4 cores to 8 cores did not halve wall time the way doubling per-core *work* would. The dispatch savings are real but small (fewer op launches), and the binary_ng side-effect turned out to matter more than the matmul speedup itself.

8 End-to-End Result and Discussion

Table 5. End-to-end device kernel time, Qwen2.5-Coder-7B decode, 4-way Blackhole TP mesh, 10 layers, 1024-context, batch 1. Full-profile totals (two trace-replay sessions), device 0.

Configuration	Total (ms)	vs. baseline
Baseline (stock recipe)	41.77	—
+ MLP HiFi2	36.75	−12.0%
+ MLP gate/up fusion	**36.38**	−12.9%
(Failed: TopK padding)	56.82	+36%

Table 5 pulls the three attempts together. The two optimizations that stuck, taken together, move end-to-end device kernel time from 41.77 ms to 36.38 ms, a 12.9% reduction. On a per- iteration basis this is roughly 10.4 ms → 9.1 ms, or a tok/s/user increase from about 96 to about 110 at 10 layers. At 2 layers (smoke-test config) the numbers were 65 tok/s/user at baseline vs 102.7 tok/s/user after both optimizations — the 2-layer picture exaggerates the speedup because TopK, unchanged by our work, is a larger fraction of per-iteration time when there are fewer layers.

What the TopK result actually means. The stock Qwen2.5-7B performance recipe on Blackhole is leaving roughly 45% of its per-token kernel time on the floor in a sampling op that is not even part of the transformer. We think this is significant, and we think it is probably not specific to Qwen: any model whose per-device vocab slice does not hit whatever threshold TopK uses for its multi-core path is likely paying a similar tax. The fact that the obvious Python-level workaround (pad the vocab) does not work means this is a genuinely kernel-level issue. A colleague working on `tt-metal` full-time would want to add a `TT_FATAL` in `reduction::topk` when `CORE COUNT` falls back to 1 on an n-element input with n above some reasonable threshold, so that future users catch this before they ship.

Why the Coder weights don't change the kernel picture. Qwen2.5-Coder-7B is architecturally identical to Qwen2.5-7B-Instruct: same layer count, hidden size, head counts, and intermediate size. So the op-level profile is identical. The only thing that changes is the weight values. We used Qwen2.5-7B-Instruct weights for most of our runs because the Coder checkpoint was not already on the host machine; by the time we could download it, the TA had signed off on the substitution. All three of our optimizations are purely kernel-level and do not depend on weight values, so they apply to Coder as well.

Compile-cache variance. During the checkpoint work we noticed that back-to-back runs of architecturally identical models showed roughly 44% variation in Matmul mean time per op, which we attributed at the time to compile-cache state. For the final report, all the comparison runs in Tables 2, 3, and 4 were taken with warm caches in the same session, and the raw CSVs are under tt-metal/generated/profiler/reports/qwen25_7b_decode_*/. The reported deltas are stable under repeated measurement.

What we did not try. We left the following on the table: (i) true TopK parallelization, which is a C++ kernel change; (ii) extending the Blackhole DRAM weight prefetcher from Llama-3.1-8B to Qwen, which would let us overlap weight fetch with compute; (iii) async AllGather + matmul to eat into the 4.8% CCL overhead. Any of these would be a multi-week effort and was outside the scope of a semester project.

9 Related Work

Fusing the gate and up projections of a SwiGLU MLP is a standard optimization on GPU-targeted inference stacks; for example, vLLM [2] and FasterTransformer both do this in their LLaMA implementations. The novelty of our version is not the algorithmic idea but the `tt-metal` implementation, in particular the observation that the binary_ng eltwise on interleaved inputs is noticeably cheaper than on width-sharded inputs. Math-fidelity tuning on Tenstorrent hardware is discussed in the `tt-metal` documentation [3]; our contribution here is a per-op measurement of what HiFi2 actually buys on the MLP matmuls for a 7B-scale SwiGLU model.

10 Conclusion

We optimized Qwen2.5-Coder-7B decode on a four-way Tenstorrent Blackhole TP mesh and reduced device kernel time by 12.9% without a generation-quality regression. The two changes that worked were straightforward: a math-fidelity drop on BFP8-weighted matmuls, and a standard SwiGLU projection fusion. The change that did not work, padding the TopK input to a power of two, was instructive. It taught us that kernel-dispatch decisions in `tt-metal` are made on tensor specs by the device-operation class, not by

the shape of the input, and that shape-based workarounds are a bad bet. The single biggest bottleneck we identified — TopK running on one core of 130 — remains as future work, because fixing it properly needs a new kernel rather than a Python-level config change.

Code. Our changes are on top of commit `a4d8480d3e` of `tt-metal`. The two optimization commits are `965e0f4622` (HiFi2) and `e05a044c4f` (gate/up fusion). `f6de95bb02` is the revert of the TopK padding attempt.

References

[1] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, Kai Dang, Yang Fan, Yichang Zhang, An Yang, Rui Men, Fei Huang, Bo Zheng, Yibo Miao, Shanghaoran Quan, Yunlong Feng, Xingzhang Ren, Xuancheng Ren, Jingren Zhou, and Junyang Lin. 2024. Qwen2.5-Coder Technical Report. arXiv:2409.12186 [cs.CL]

[2] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*. ACM. https://doi.org/10.1145/3600006.3613165

[3] Tenstorrent. 2026. TT-Metalium: Tenstorrent's Low-Level Programming Model and SDK. https://github.com/tenstorrent/tt-metal. Accessed April 2026.